

DDGAN: Adversarial Robustness Training for Deepfake Detection

A Technical Report

Abstract

This report presents a comprehensive technical description of DDGAN, a framework for training adversarially robust deepfake detectors. Unlike classical GANs that synthesize realistic images, DDGAN employs a generator that produces **bounded adversarial perturbations** while the discriminator maintains strong classification capability under these perturbations. The training objective combines consistency-based robustness with margin-based adversarial pressure, resulting in discriminators that generalize better to unseen deepfake generation methods.

Table of Contents

- [1. Introduction](#)
- [2. Data Pipeline](#)
- [3. Architecture](#)
- [4. Loss Functions](#)
- [5. Optimization](#)
- [6. Training Algorithm](#)
- [7. Evaluation Metrics](#)
- [8. Hyperparameters](#)

1. Introduction

1.1 Problem Statement

Deepfake detection models often overfit to artifacts specific to particular generation methods, leading to poor generalization. Adversarial robustness training addresses this by forcing the discriminator to classify correctly even under adversarial perturbations.

1.2 Key Insight

The generator in DDGAN does **not** synthesize fake images. Instead, it produces small, bounded perturbations δ that are added to real images:

$$x_{adv} = x + \delta, \quad \|\delta\|_{\infty} \leq \epsilon$$

The discriminator is trained to produce **consistent predictions** between x and x_{adv} .

2. Data Pipeline

2.1 Dataset Structure

```
celebdfv2_images/  
├── train/  
│   ├── real/      # Real face images (label = 1)  
│   └── fake/      # Deepfake images (label = 0)  
└── test/  
    ├── real/  
    └── fake/
```

2.2 Label Convention

Class	Label	Description
Real	1	Authentic face images (positive class)
Fake	0	Deepfake images (negative class)

This convention is normalized across both local and HuggingFace data sources.

2.3 Data Augmentation

Training transforms:

$$\mathcal{T}_{train} = \text{HFlip}(p=0.5) \circ \text{ColorJitter} \circ \text{Rotate}(\pm 10^\circ) \circ \text{Normalize}(\mu, \sigma)$$

Where:

- $\mu = (0.485, 0.456, 0.406)$ (ImageNet mean)
- $\sigma = (0.229, 0.224, 0.225)$ (ImageNet std)

Validation transforms:

$$\mathcal{T}_{val} = \text{Normalize}(\mu, \sigma)$$

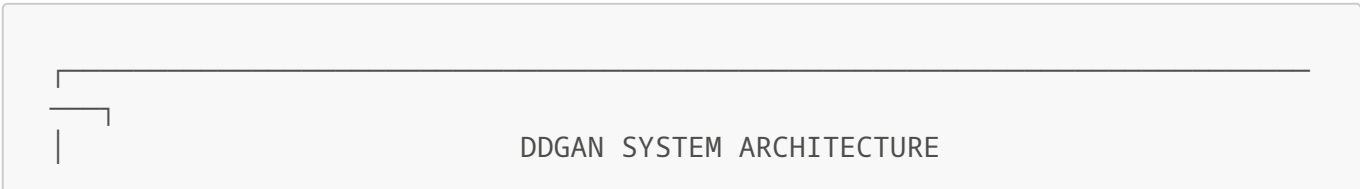
2.4 Class Imbalance Handling

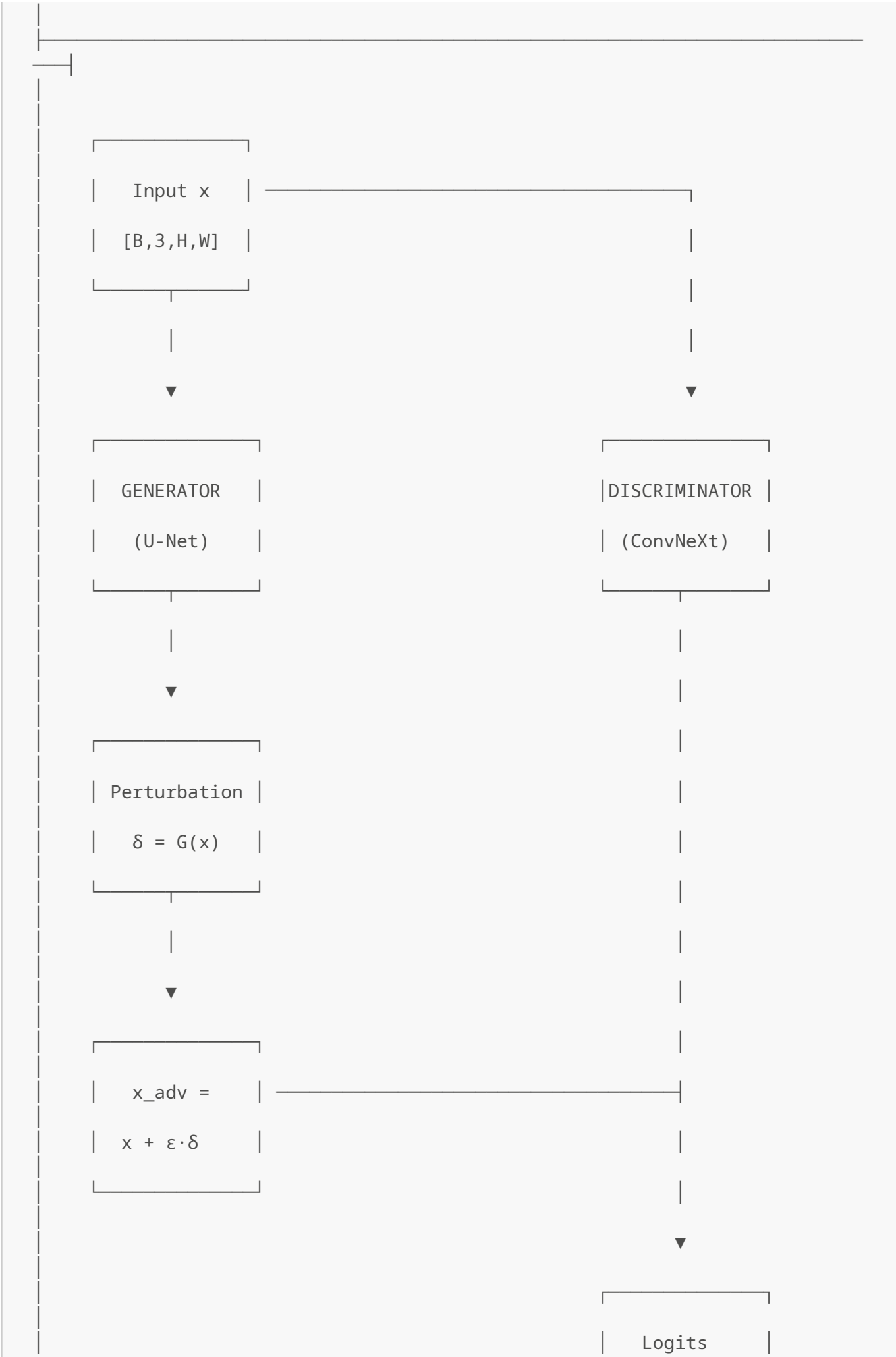
The dataset exhibits class imbalance (~88% fake, ~12% real). This is addressed via weighted BCE:

$$\text{pos_weight} = \sqrt{\frac{N_{fake}}{N_{real}}} \approx 3.0$$

3. Architecture

3.1 System Overview



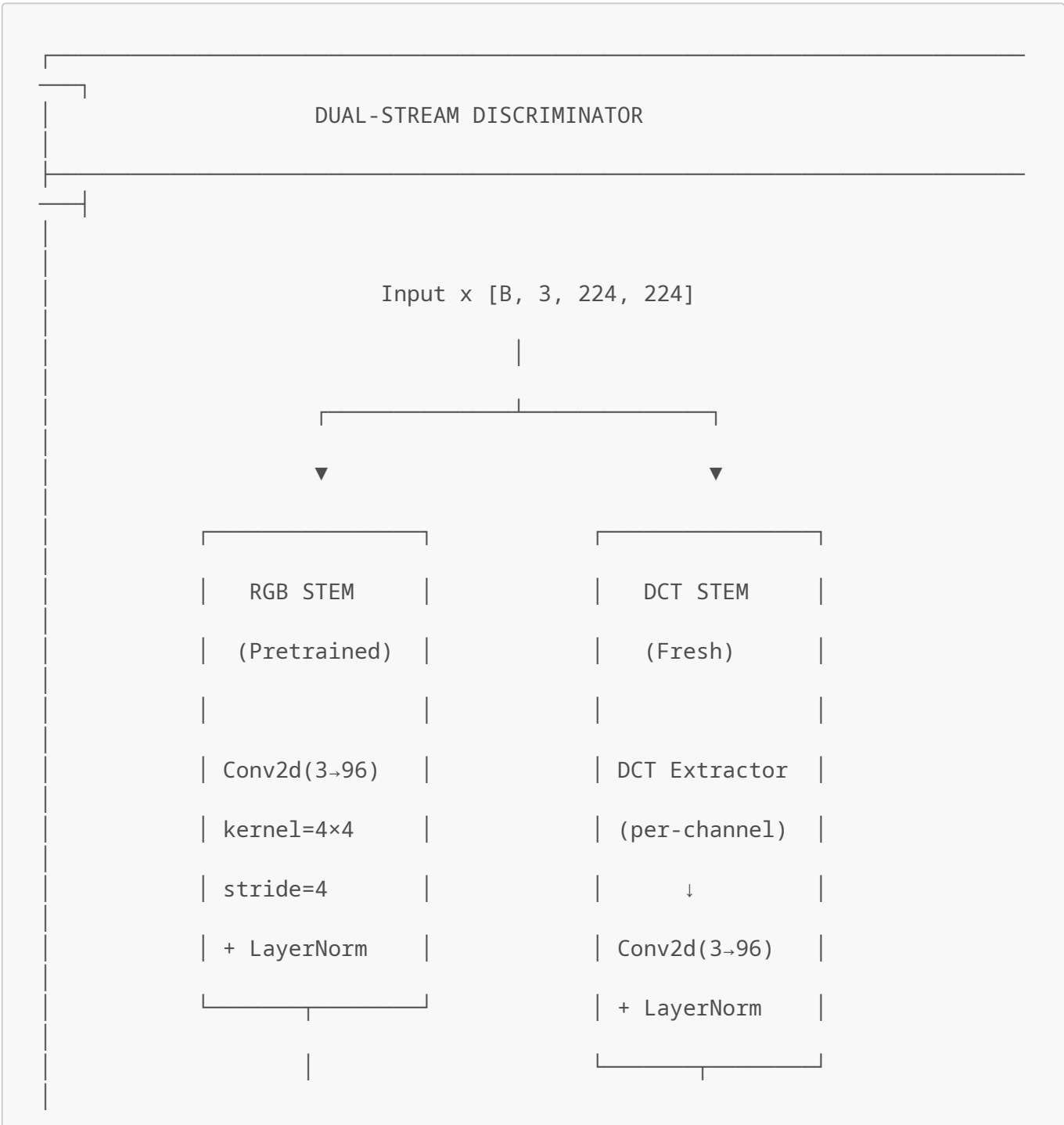


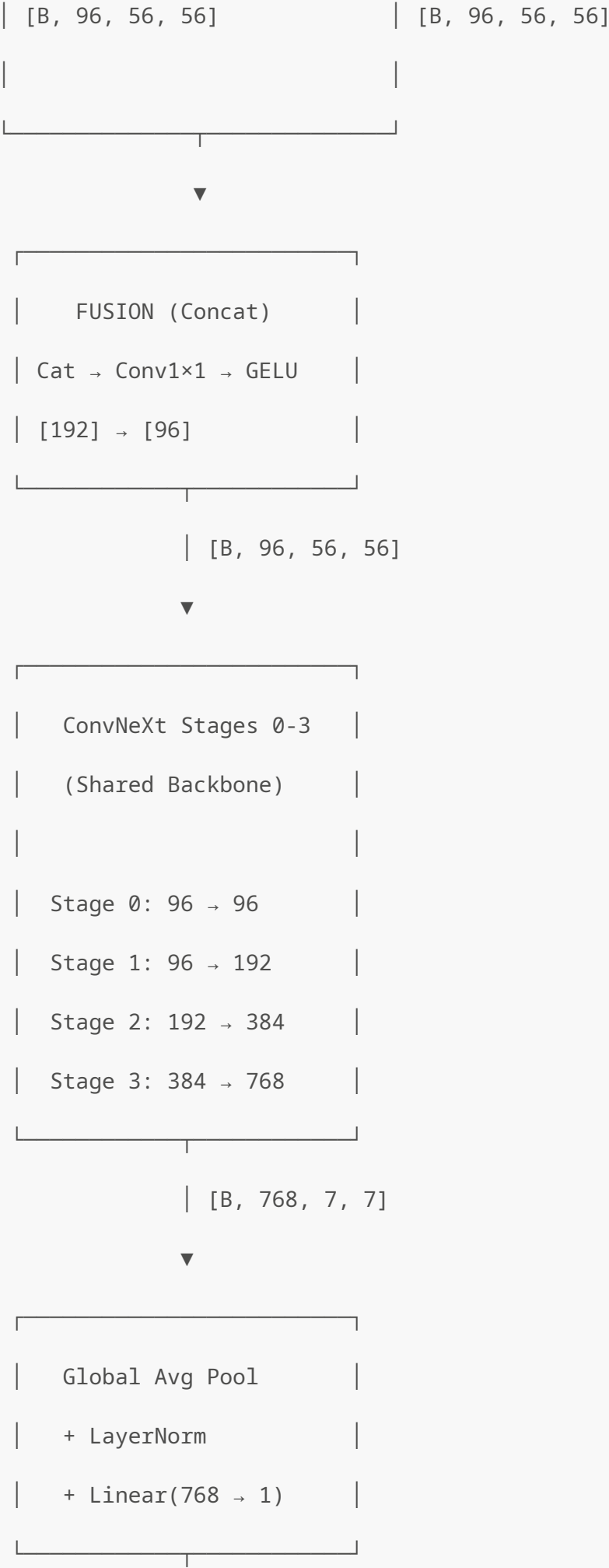
| D(x_adv) |

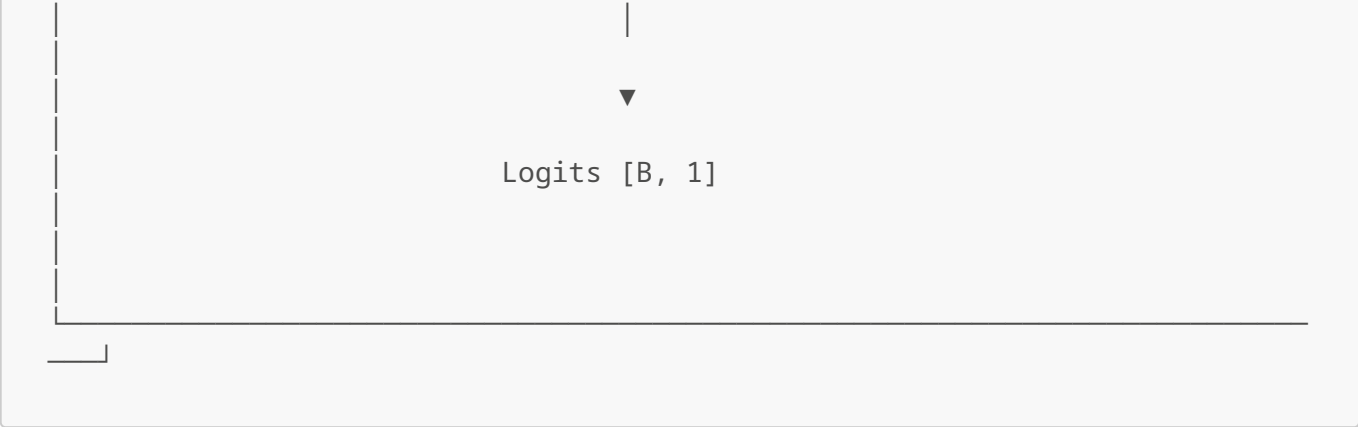
|

3.2 Discriminator Architecture

3.2.1 Dual-Stream Design (Default)







3.2.2 DCT Feature Extraction

The 2D Discrete Cosine Transform (DCT-II) is defined as:

$$F(u, v) = \alpha(u) \alpha(v) \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) \cos\left[\frac{\pi(2m+1)u}{2M}\right] \cos\left[\frac{\pi(2n+1)v}{2N}\right]$$

Where:

$$\alpha(k) = \begin{cases} \sqrt{1/N} & k = 0 \\ \sqrt{2/N} & k > 0 \end{cases}$$

Per-channel mode: DCT is applied to each RGB channel independently, producing a 3-channel frequency representation.

Learnable frequency filters: A learnable mask $\mathbf{M} \in \mathbb{R}^{3 \times H \times W}$ modulates frequency bands:

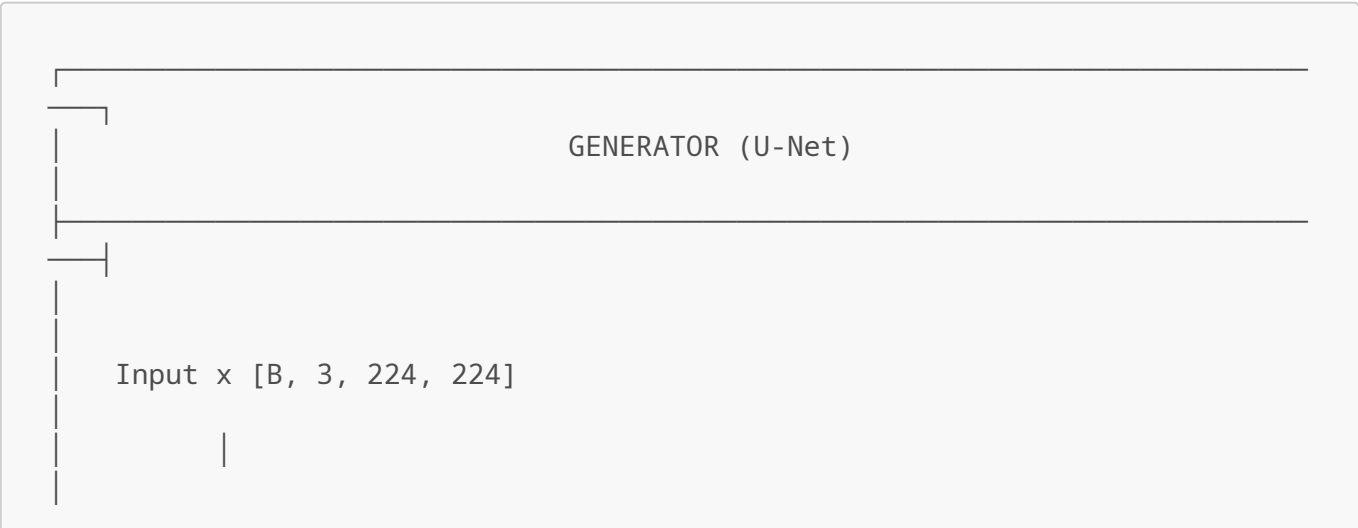
$$\hat{F}_c(u, v) = M_c(u, v) \cdot F_c(u, v)$$

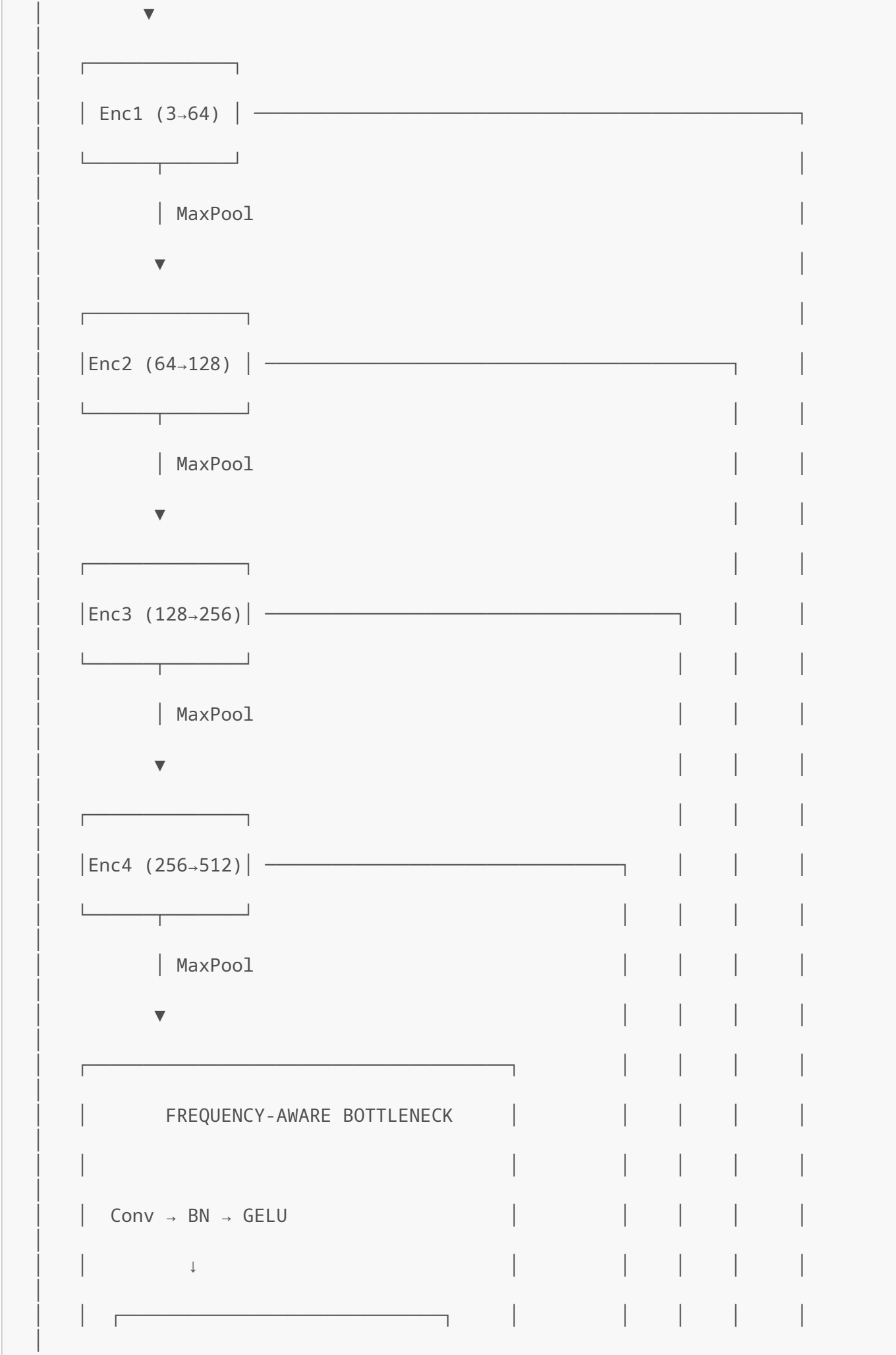
Log scaling: For numerical stability:

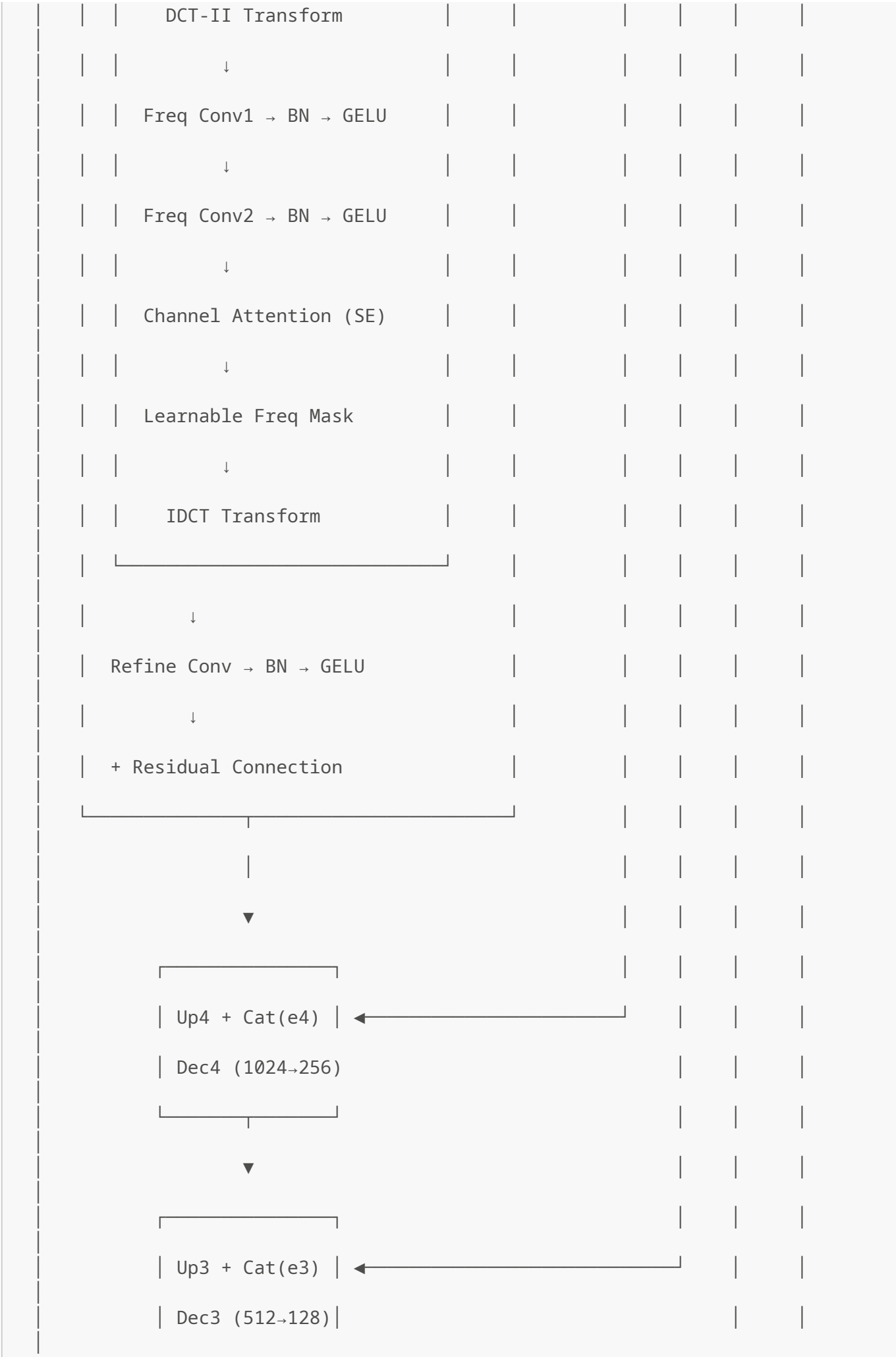
$$\tilde{F}(u, v) = \log(|F(u, v)| + \epsilon), \quad \epsilon = 10^{-6}$$

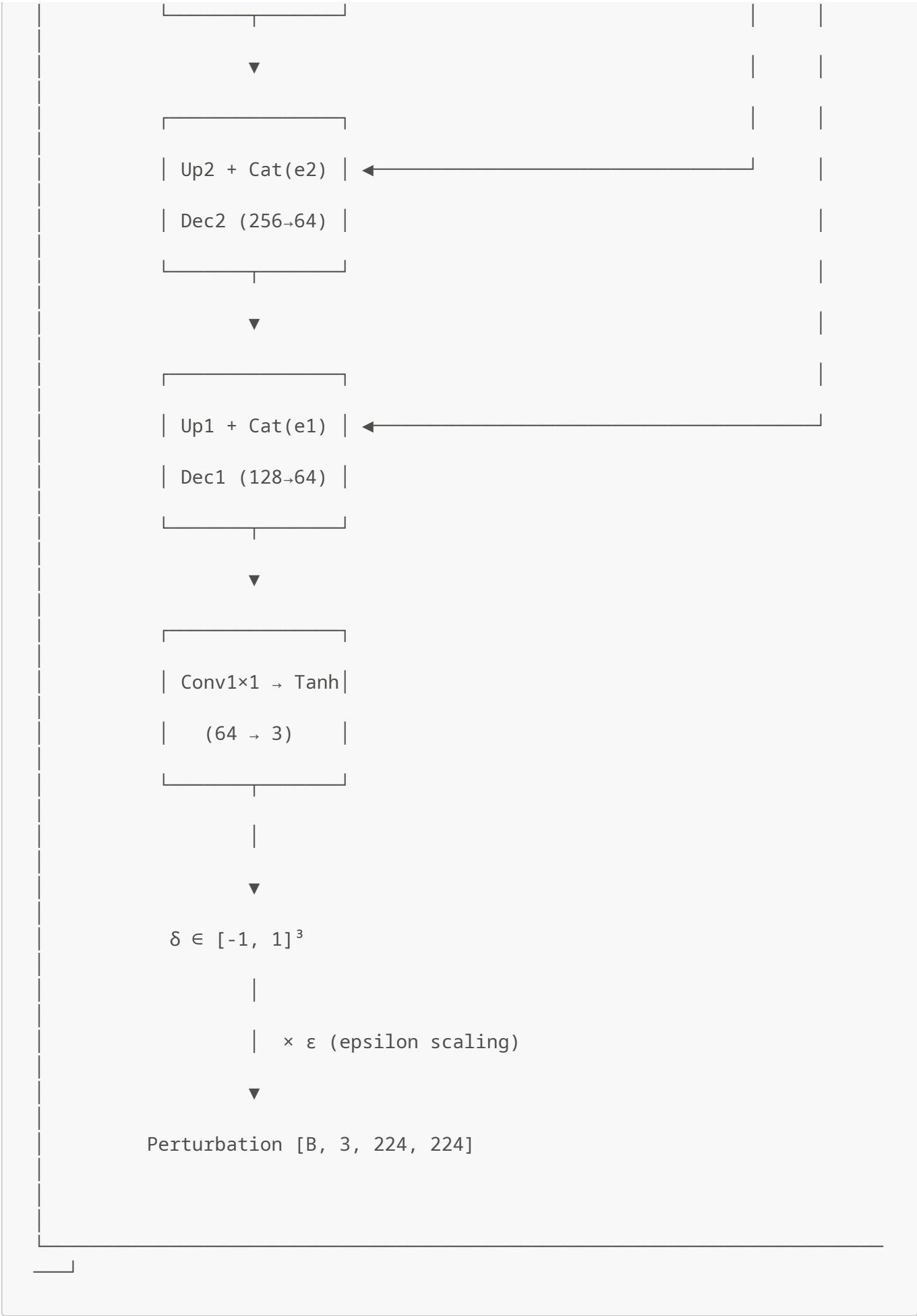
3.3 Generator Architecture

3.3.1 U-Net with Frequency-Aware Bottleneck









3.3.2 Epsilon Constraint

The perturbation is bounded:

$$\|\delta\|_2 = \epsilon \cdot \tanh(G_{\text{raw}}(x)), \quad \|\delta\|_2 \leq \epsilon$$

Default: $\epsilon = 0.03$ (approximately 7.65/255 in pixel space).

4. Loss Functions

4.1 Discriminator Loss

The discriminator loss consists of three components:

$$\mathcal{L}_D = \mathcal{L}_{\text{real}} + \mathcal{L}_{\text{fake}} + w_{\text{cons}}(t) \cdot \lambda_{\text{cons}} \cdot \mathcal{L}_{\text{cons}}$$

4.1.1 Classification Losses (BCE with Logits)

$$\mathcal{L}_{\text{real}} = -\frac{1}{N_r} \sum_{i=1}^{N_r} \left[w_+ \log(\sigma(D(x_i^r))) \right]$$

$$\mathcal{L}_{\text{fake}} = -\frac{1}{N_f} \sum_{i=1}^{N_f} \left[\log(1 - \sigma(D(x_i^f))) \right]$$

Where:

- $\sigma(\cdot)$ is the sigmoid function
- $w_+ = 3.0$ is the positive class weight (handles imbalance)

4.1.2 Consistency Loss

The consistency loss penalizes **prediction drift** under adversarial perturbation:

$$\mathcal{L}_{\text{cons}} = \frac{1}{N_r} \sum_{i=1}^{N_r} \left(\sigma(D(x_i^r)) - \sigma(D(x_i^{\text{adv}})) \right)^2$$

Key insight: This operates in **probability space** (after sigmoid), preserving the ranking relationship.

4.1.3 Consistency Warmup

To prevent early training collapse, consistency weight ramps up:

$$w_{\text{cons}}(t) = \min\left(1, \frac{t}{T_{\text{warmup}}}\right), \quad T_{\text{warmup}} = 5 \text{ epochs}$$

4.2 Generator Loss

The generator loss combines margin-based adversarial pressure with perturbation regularization:

$$\mathcal{L}_G = \mathcal{L}_{\text{margin}} + \lambda_{\text{pert}} \cdot \mathcal{L}_{\text{pert}}$$

4.2.1 Margin-Based Loss

Unlike standard GAN losses that push logits to extremes, the margin loss provides **bounded pressure**:

$$\mathcal{L}_{margin} = \frac{1}{N_r} \sum_{i=1}^{N_r} \frac{\max(0, D(x_i^{adv}) - m)}{\overline{|D(x^{adv})|}_{stop} + \epsilon}$$

Where:

- m is the margin threshold (default: 0.3)
- The denominator normalizes by mean absolute logit (with stop-gradient)
- $\epsilon = 10^{-6}$ for numerical stability

Behavior:

- If $D(x_{adv}) > m$: Generator is penalized (discriminator still confident)
- If $D(x_{adv}) \leq m$: No penalty (generator succeeded)

This prevents unbounded logit collapse.

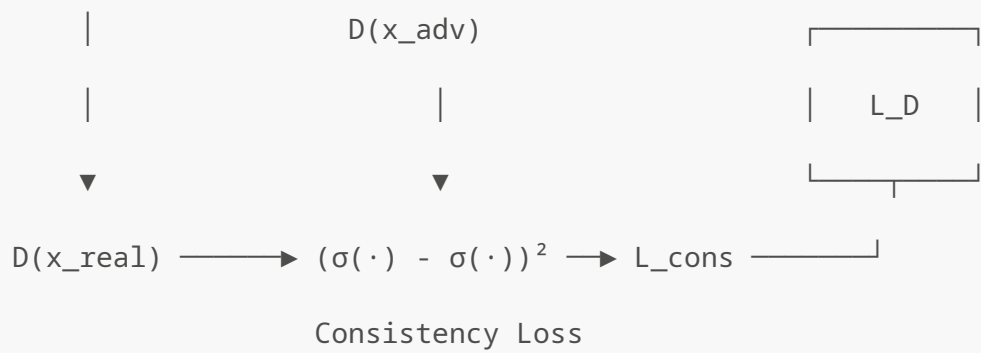
4.2.2 Perturbation Regularization

$$\mathcal{L}_{pert} = \frac{1}{N_r \cdot C \cdot H \cdot W} \sum \delta^2$$

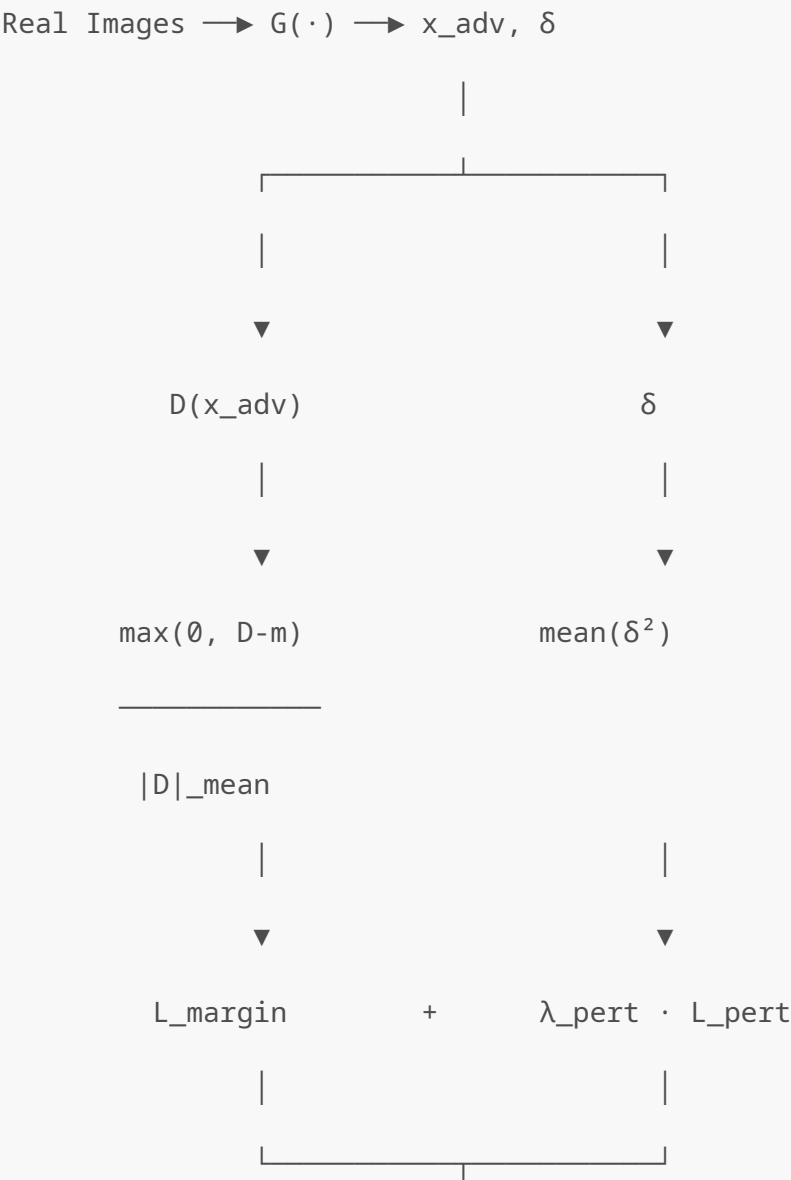
L2 regularization encourages smooth, minimal perturbations.

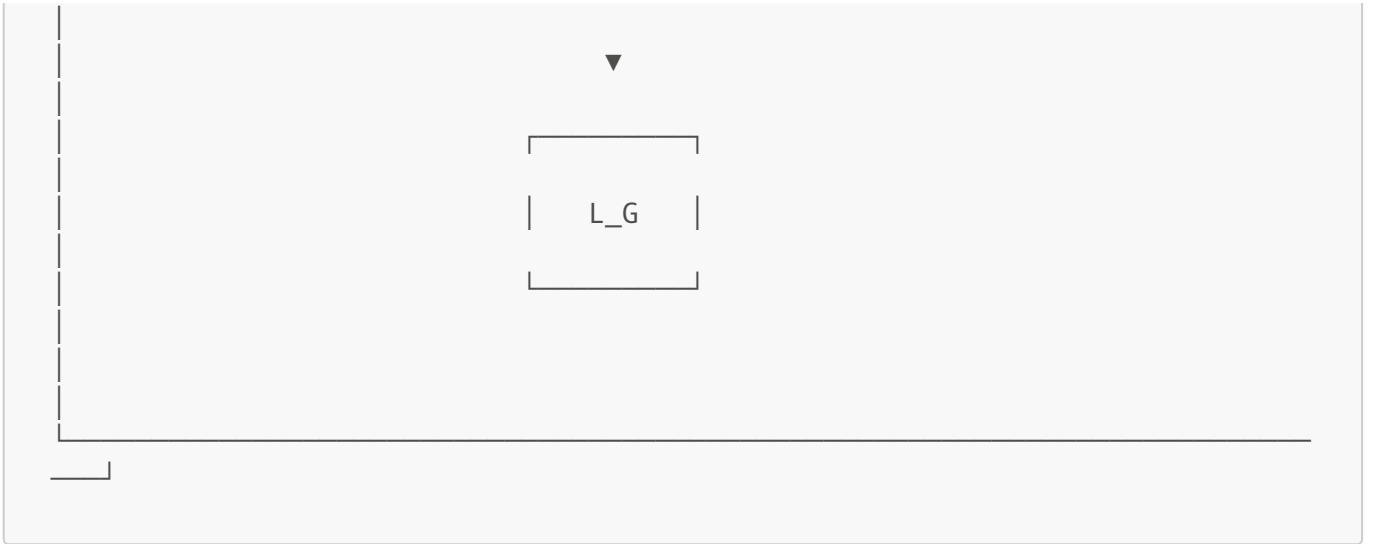
4.3 Loss Diagram





GENERATOR LOSS





5. Optimization

5.1 Optimizers

Separate AdamW optimizers for discriminator and generator:

Discriminator: $\theta_D \leftarrow \theta_D - \alpha_D \cdot \text{AdamW}(\nabla_{\theta_D} \mathcal{L}_D)$

Generator: $\theta_G \leftarrow \theta_G - \alpha_G \cdot \text{AdamW}(\nabla_{\theta_G} \mathcal{L}_G)$

Where:

- $\alpha_D = \alpha_{\text{base}} \cdot \text{d_lr_mult} = 10^{-4} \cdot 0.2 = 2 \cdot 10^{-5}$
- $\alpha_G = \alpha_{\text{base}} \cdot \text{g_lr_mult} = 10^{-4} \cdot 0.5 = 5 \cdot 10^{-5}$
- $\beta_1 = 0.5$, $\beta_2 = 0.999$
- Weight decay: 10^{-2}

5.2 Learning Rate Schedule

Cosine annealing with minimum LR:

$$\alpha(t) = \alpha_{\min} + \frac{1}{2}(\alpha_{\max} - \alpha_{\min}) \left(1 + \cos \left(\frac{t \cdot \pi}{T_{\max}} \right) \right)$$

Where $\alpha_{\min} = 10^{-6}$.

5.3 Gradient Clipping

Both networks use gradient clipping:

$$\hat{g} = \frac{g}{\max(1, \|g\|_2 / \tau)}, \quad \tau = 1.0$$

5.4 Mixed Precision Training

Uses PyTorch AMP with 16-bit mixed precision for memory efficiency and speed.

6. Training Algorithm

6.1 Training Loop (Pseudocode)

Algorithm: DDGAN Training

Input: Dataset D , epochs T , consistency_weight λ_c , margin m

Output: Trained discriminator D_θ , generator G_ϕ

```

for epoch  $t = 1$  to  $T$  do
     $w_{\text{cons}} \leftarrow \min(1, t / 5)$     // Consistency warmup

    for batch  $(x, y)$  in  $D$  do
        // Separate real and fake samples
         $x_{\text{real}} \leftarrow x[y == 1]$ 
         $x_{\text{fake}} \leftarrow x[y == 0]$ 

        // ===== DISCRIMINATOR UPDATE =====
        // Classification losses
         $L_{\text{real}} \leftarrow \text{BCE}(D(x_{\text{real}}), 1)$ 
         $L_{\text{fake}} \leftarrow \text{BCE}(D(x_{\text{fake}}), 0)$ 

        // Generate adversarial samples
         $x_{\text{adv}}, \delta \leftarrow G(x_{\text{real}})$ 

        // Consistency loss (detach gradients appropriately)
         $L_{\text{cons}} \leftarrow \text{MSE}(\sigma(D(x_{\text{real}})).\text{detach}(), \sigma(D(x_{\text{adv}}).\text{detach}()))$ 

        // Total discriminator loss
         $L_D \leftarrow L_{\text{real}} + L_{\text{fake}} + w_{\text{cons}} \cdot \lambda_c \cdot L_{\text{cons}}$ 

        // Update discriminator
         $\theta_D \leftarrow \theta_D - \alpha_D \cdot \nabla L_D$ 

        // ===== GENERATOR UPDATE =====
        // Fresh forward pass (after D update)
         $x_{\text{adv}}, \delta \leftarrow G(x_{\text{real}})$ 
         $\text{logits}_{\text{adv}} \leftarrow D(x_{\text{adv}})$ 

        // Margin loss (bounded adversarial pressure)
         $L_{\text{margin}} \leftarrow \text{mean}(\text{ReLU}(\text{logits}_{\text{adv}} - m) /$ 
         $(|\text{logits}_{\text{adv}}|. \text{mean}().\text{detach}() + \epsilon))$ 

        // Perturbation regularization
         $L_{\text{pert}} \leftarrow \text{mean}(\delta^2)$ 

        // Total generator loss
         $L_G \leftarrow L_{\text{margin}} + \lambda_{\text{pert}} \cdot L_{\text{pert}}$ 

        // Update generator
         $\phi_G \leftarrow \phi_G - \alpha_G \cdot \nabla L_G$ 

```

```

    end for

    // Step learning rate schedulers
    scheduler_D.step()
    scheduler_G.step()
end for

```

6.2 Update Order

Critical: The discriminator is updated **before** the generator in each iteration. The generator then produces a fresh adversarial sample against the updated discriminator. This prevents stale gradients and ensures proper adversarial pressure.

7. Evaluation Metrics

7.1 Clean Metrics

Standard classification metrics on unperturbed test samples:

- **Accuracy:** $\frac{TP + TN}{N}$
- **Precision:** $\frac{TP}{TP + FP}$
- **Recall:** $\frac{TP}{TP + FN}$
- **F1 Score:** $\frac{2 \cdot P \cdot R}{P + R}$
- **AUC-ROC (clean):** Area under ROC curve for $\{(x, y)\}_{test}$

7.2 Adversarial Metrics

Metrics on adversarially perturbed test samples:

- **AUC-ROC (adversarial):** Area under ROC curve for $\{(x + G(x), y)\}_{test}$

7.3 Robustness Gap

The primary robustness metric:

$$\text{Robustness Gap} = \text{AUC}_{clean} - \text{AUC}_{adv}$$

Interpretation:

- Lower gap $\rightarrow$ more robust model
- Negative gap $\rightarrow$ impossible (adversarial shouldn't help)
- Gap $\rightarrow 0 \rightarrow$ perfect robustness

7.4 Early Stopping

Two early stopping criteria:

1. **AUC-based:** Stop if `val_auc_adv` doesn't improve for 10 epochs
2. **Gap-based:** Stop if robustness gap changes by < 0.002 for 3 consecutive epochs

8. Hyperparameters

8.1 Summary Table

Category	Parameter	Default	Description
Data	batch_size	32	Samples per batch
	image_size	224	Input resolution
	num_workers	4	DataLoader workers
Model	d_backbone	convnext_tiny	Discriminator backbone
	d_type	dual_stream	Discriminator architecture
	d_fusion_type	concat	How to fuse RGB and DCT
	g_base_channels	64	Generator base width
	epsilon	0.03	Max perturbation magnitude
	consistency_weight	1.0	Weight for $\mathcal{L}_{\text{cons}}$
Loss	margin	0.3	Margin threshold in $\mathcal{L}_{\text{margin}}$
	perturb_weight	0.02	Weight for $\mathcal{L}_{\text{pert}}$
	pos_weight	3.0	BCE positive class weight
	learning_rate	1e-4	Base learning rate
Optim	d_lr_mult	0.2	Discriminator LR multiplier
	g_lr_mult	0.5	Generator LR multiplier
	betas	(0.5, 0.999)	Adam momentum terms
	weight_decay	0.01	L2 regularization
	gradient_clip	1.0	Max gradient norm
	max_epochs	30	Maximum training epochs
Train	precision	16-mixed	Mixed precision mode

8.2 Hyperparameter Sensitivity

Critical parameters:

- margin**: Too high → generator ineffective; too low → logit collapse
- consistency_weight**: Too high → discriminator becomes passive
- epsilon**: Too high → perturbations become visible; too low → no adversarial pressure

Appendix A: Mathematical Notation

Symbol	Description
x	Clean input image
x_{adv}	Adversarially perturbed image
δ	Perturbation tensor
D	Discriminator (outputs logits)
G	Generator (outputs perturbation)
σ	Sigmoid function
ϵ	Perturbation bound
m	Margin threshold
λ	Loss weight hyperparameters
θ_D, ϕ_G	Network parameters
α	Learning rate

Appendix B: File Reference

File	Purpose
config.py	Configuration dataclasses
train.py	Training script with CLI
lightning_module.py	PyTorch Lightning module
models/discriminator.py	Discriminator architectures
models/generator.py	U-Net generator
models/dct_extractor.py	DCT feature extraction
data/datamodule.py	Data loading and augmentation